Indian Institute of Technology Palakkad

ID1110 Introduction to Programming

End Semester Exam (29 Apr 2026)

Time: 9:30 hrs to 11:30 hrs Points: 40

Name: .....

Roll No: .....

Instructions

1. This quiz-booklet has 6 pages containing 6 questions. Please get a new booklet if any question/page is missing.
2. Write your **name** and **roll number** in the space above.
3. Answer each question in the corresponding space provided. **Additional sheets may be used only for rough work and will not be evaluated.** Do not attach the rough sheets to this booklet.
4. Write your answers neatly in blue or black ink. Do not use pencil or red ink.
5. Do not keep any books, papers, mobile phones or other smart gadgets with you during the exam.

Space for marks and feedback (do not write anything below)

Question	1	2	3	4	5	6	Total
Marks							

1. For non-negative integers n and k with $k \leq n$, $\binom{n}{k}$ denotes the number of k-sized subsets of an n-sized set. These numbers are called binomial coefficients and have the following recursive definition $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ for positive integers n, k satisfying $1 \leq k < n$ with base cases $\binom{n}{0} = \binom{n}{n} = 1$ for all integers $n \geq 0$. Complete the following recursive function definition that computes $\binom{n}{k}$. [5]

```
def binomial(n, k):
    if k < 0 or k > n:
        return 0
    if k == 0 or k == n:
        return 1
    return binomial(n-1,k-1)+binomial(n-1,k)
```

Now, complete the following snippet that reduces the number of recursive calls while computing binomial coefficients.

```
bcs = {}
def binomial(n, k):
    if k < 0 or k > n:
        return 0
    if k == 0 or k == n:
        return 1
    if (n, k) in bcs:
        return bcs[(n,k)]
    bcs[(n,k)]=Binomial(n-1,k-1)+Binomial(n-1,k)
    return bcs[(n,k)]
```

2. Consider the problem of approximating a sequence of values $x_1, x_2, \ldots, x_n$ by a sequence of estimates $y_1, y_2, \ldots, y_n$ where we need to compute mean square error defined as $\frac{1}{n}\sum_{i=1}^{n}(x_i - y_i)^2$. Complete the following code that does this computation. [4]

```
import numpy as _____
predictions = np.array([1.5,2.3,3.7])
values = np.array([1,2.5,3])
n = len(predictions)
d = predictions-values
sq = np.square(d)
mse = np.sum(sq)/n
print(mse)
```

3. Complete the following code that opens a file named story.txt in read mode, reads its content and writes it into a new file named story1.txt after replacing every occurrence of the word Sherlock with Holmes. [6]

```
try:
    with open("story.txt","r") as f:
        content = f.read()
    updated = content.replace("Sherlock", "Holmes")
    with open("story1.txt","w") as f:
        f.write(updated)
except FileNotFoundError:
    print("Error:  story.txt not found")
except PermissionError:
    print("Error:  permission denied")
except Exception as e:
    print("An unexpected error occurred:", e)
```

4. Consider the following code. [10]

```
characters = [
    {"name": "Harry Potter", "house": "Gryffindor", "wand": "Holly"},
    {"name": "Hermione Granger", "house": "Gryffindor", "wand": "Vine"},
    {"name": "Ron Weasley", "house": "Gryffindor", "wand": "Willow"},
    {"name": "Draco Malfoy", "house": "Slytherin", "wand": "Hawthorn"}
]
gryf = [c for c in characters if c["house"] == "Gryffindor"]
houses = {
    "Gryffindor": ["Harry Potter", "Hermione Granger", "Ron Weasley"],
    "Slytherin": ["Draco Malfoy"],
    "Ravenclaw": ["Luna Lovegood"],
    "Hufflepuff": ["Cedric Diggory"]
}
uh = {c["house"] for c in characters}
```

Give the output for each of the following statements.

1. `print(characters[3]["wand"])`
   Hawthorn
2. `print(houses["Gryffindor"])`
   ['Harry Potter', 'Hermino Granger', 'Ron Weasley']
3. `print(houses["Slytherin"][0])`
   Draco Malfoy
4. `print(characters[2]["name"])`
   Ron Weasley

5. `names = [c["name"] for c in gryf if c["wand"] != "Willow"]`
   `print(names)`
   ['Harry Potter', 'Hermino Granger']

6. `print(", ".join(c["name"] for c in gryf if c["wand"] != "Willow"))`
   Harry Potter,Hermino Granger

7. `print(uh)`
   {'Slytherin', 'Gryffindor'}

8. `print(type(uh))`
   <class 'set'>

9. `print(type(characters))`
   <class 'list'>

10. `print(type(houses))`
    <class 'dict'>

5. There are 52 playing cards in a standard deck – each of them belongs to one of four suits and one of thirteen ranks. The suits are Spades, Hearts, Diamonds, and Clubs. The ranks are Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, and King. The definition for a class that represents a playing card is given below. The attributes of a new card object are `rank` and `suit` and these are integers (for ease of comparison). The suits and the corresponding integers are: Spades `3`, Hearts `2`, Diamonds `1`, Clubs `0`. To represent the ranks, we use the integer `2` to represent the rank `2`, `3` to represent `3`, and so on up to `10`. And we use `1` to represent an Ace. The other face cards and the corresponding integers are: Jack `11`, Queen `12`, King `13`. Cards are compared first by suit and then by rank. Here's a definition for a class that represents a playing card. As an example usage, `queen = Card(1, 12)` creates a card object denoting Queen of Diamonds. [10]

```
class Card:

    suit_names = ['Clubs', 'Diamonds', 'Hearts', 'Spades']
    rank_names = [None, 'Ace', '2', '3', '4', '5', '6', '7', '8', '9', '10',
                  'Jack', 'Queen', 'King']

    def __init__(self, suit, rank):
        self.suit = suit
        self.rank = rank

    def __str__(self):
        rank_name = Card.rank_names[self.rank]
        suit_name = Card.suit_names[self.suit]
        return f'{rank_name} of {suit_name}'

    def __eq__(self, other):
        return self.suit == other.suit and self.rank == other.rank
```

```
def to_tuple(self):
    return (self.suit, self.rank)

def __lt__(self, other):
    return self.to_tuple() < other.to_tuple()

def __le__(self, other):
    return self.to_tuple() <= other.to_tuple()
```

Note the following points.

1. Tuple comparison compares the first elements from each tuple and if they are the same, it compares the second elements.
2. If we use the `!=` operator, Python invokes `__ne__` method, if it exists. Otherwise it invokes `__eq__` and negates the result.
3. If we use the `>` operator, Python invokes `__gt__` if is defined. Otherwise it invokes `__lt__` with the arguments in the opposite order. Similar behaviour holds for `__ge__` and `__le__`.

Give the output for each of the following snippets.

(a) `print(Card.suit_names[2])`

Hearts

(b) `print(Card.rank_names[13])`

King

(c)
```
x = Card(2, 10)
print(x.suit, x.rank)
```

2 10

(d) `print(x)`

10 of Hearts

(e)
```
y = Card(3, 12)
print(x == y)
```

Queen of Spades

False

(f)
```
z = Card(0, 13)
print(z)
```

King of Clubs

(g) `print(x == z)`

False

(h) `print(x < z)`

False

(i) `print(z >= x)`

False

(j) `print(x != z)`

True

6. Consider the following class definitions for `Deck` and `FilteredDeck`. [5]

```
import random

class Deck:

    def __init__(self):
        self.cards = [Card(s, r) for s in range(len(Card.suit_names))
                      for r in range(1, 14)]

    def __str__(self):
        return '\n'.join(str(card) for card in self.cards)

    def shuffle(self):
        random.shuffle(self.cards)

    def sort(self):
        self.cards.sort()

class FilteredDeck(Deck):

    def __init__(self, suits):
        super().__init__()
        self.cards = [card for card in self.cards if card.suit in suits]
```

Give the output for each of the following snippets.

(a)
```
deck = Deck()
print(len(deck.cards))
```
4 suits × 13 ranks = 52 cards

52

Sorted by (suit, rank) → starts at (0,1), (0,2), (0,3) → first 3 are lowest ranked Clubs

(b)
```
deck.sort()
for card in deck.cards[:4]:
    print(card, end=',')
```
Ace of Clubs,2 of Clubs,3 of Clubs,4 of Clubs,

keep only suites 1 (Diamonds) and 2 (Hearths)

(c)
```
deck1 = FilteredDeck([1,2])
print(len(deck1.cards))
```
26

(d)
```
deck1.sort()
for card in deck1.cards[:3]:
    print(card, end=',')
```
sorted start at (1,1) , (1,2), (1,3)

Ace of Diamonds,2 of Diamonds,3 of Diamonds,

(e)
```
deck2 = FilteredDeck([1])
print(len(deck2.cards))
```
13